

Explicación RTL del sistema star_tracker_top

Documentación generada desde Docs/EXPLICACION_SRC_RTL.md

Proyecto: tesis-star-tracker

Alcance de este documento: solo archivos RTL. No se documentan testbenches.

Flujo completo del sistema:

I2C/SCCB init -> XCLK -> captura OV7670 -> extracción Y -> BRAM -> UART

Índice

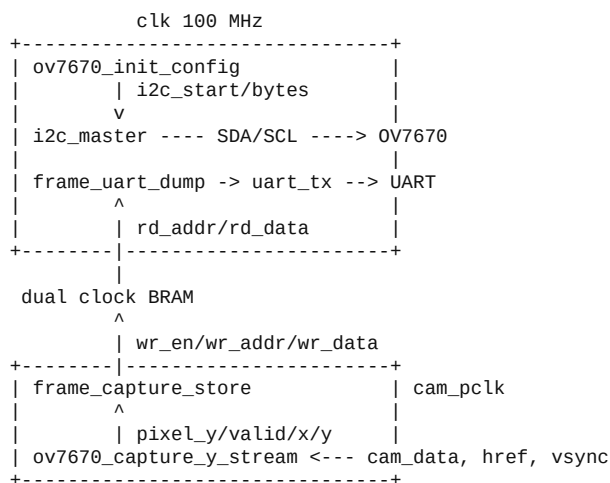
1. Vista general del sistema
2. i2c_master
3. uart_tx
4. ov7670_init_config
5. ov7670_xclk_gen
6. ov7670_capture_y_stream
7. frame_capture_store
8. framebuffer_y_bram
9. frame_uart_dump
10. star_tracker_top
11. Conclusión: recorrido completo de una imagen

1. Vista general del sistema

El diseño configura una cámara OV7670 por SCCB/I2C, le entrega un reloj externo `cam_xclk`, recibe el video paralelo por `cam_pclk`, `cam_vsync`, `cam_href` y `cam_data[7:0]`, extrae solo bytes de luminancia Y, guarda un frame QQVGA completo en BRAM y después lo envía por UART.

La configuración esperada para la cámara es QQVGA 160x120, YUV 4:2:2, orden de bytes Y U Y V. Por eso el capturador no guarda todos los bytes del bus: guarda solo los bytes en fase 0 y fase 2.

Diagrama global:



ov7670_xclk_gen: clk 100 MHz -> cam_xclk 24 MHz hacia la cámara

2. Módulo i2c_master

Archivo: Protocolo de comunicacion/rtl/i2c_master.vhd

Función general

Es un maestro I2C/SCCB básico. Genera SCL, maneja SDA como línea open-drain y permite una transacción de escritura de 1 o 2 bytes, o una lectura de 1 byte. En este proyecto se usa principalmente para escribir pares (registro, valor) a la OV7670.

Lugar dentro del sistema

I2C / SCCB. Está conectado al controlador ov7670_init_config.

Entradas principales

Señal	Descripción
clk	Reloj del sistema, 100 MHz en el top.
rst	Reset síncrono del módulo.
start	Pulso/orden de inicio de transacción.
slave_addr	Dirección I2C de 7 bits. En el top: "0100001".
rw	0 escritura, 1 lectura.
tx_byte0, tx_byte1	Datos a transmitir.
tx_count	Cantidad de bytes de escritura: 1 o 2.
SDA	Línea bidireccional I2C.

Salidas principales

Señal	Descripción
SCL	Reloj I2C generado internamente.
rx_data	Byte leído cuando rw= '1'.
busy	Transacción en curso.
done	Pulso de fin de transacción.
ack_error	Se activa si el esclavo no responde ACK o si tx_count no es válido.

Generics importantes

Generic	Valor por defecto	Uso
CLK_FREQ_HZ	100000000	Frecuencia de clk.
I2C_SCL_HZ	100000	Frecuencia objetivo de SCL.

$TICK_DIVIDER = CLK_FREQ_HZ / (I2C_SCL_HZ * 2)$. La FSM avanza en ticks de medio periodo de SCL.

Dominio de reloj

Todo el módulo trabaja en clk.

Reset

Con rst= '1' vuelve a IDLE, libera SDA, pone SCL= '1', limpia busy, done, ack_error, registros de desplazamiento, rx_reg y bit_index.

Funcionamiento paso a paso

1. En IDLE, si start='1', captura slave_addr, rw, tx_count, carga shift_reg <= slave_addr & rw y empieza condición START.
2. Baja SDA mientras SCL está alto.
3. Baja SCL y comienza a preparar bits.
4. Transmite bits MSB primero con dos estados: preparar dato con SCL=0, luego subir SCL.
5. Después de cada byte libera SDA para leer ACK del esclavo.
6. Si el ACK falla (SDA='1'), marca ack_error y va a STOP.
7. Si era escritura, después de la dirección manda tx_byte0 y opcionalmente tx_byte1.
8. Si era lectura, después del ACK de dirección lee 8 bits y manda NACK.
9. Genera STOP: SDA sube mientras SCL está alto.
10. Pulsa done y espera que start vuelva a 0 antes de regresar a IDLE.

Diagrama de estados

```

IDLE --start--> START_SDA_LOW -> START_SCL_LOW -> PREPARE_DATA_LOW
      ^                                     |
      |                                     v
WAIT_START_RELEASE <- STOP_RELEASE <- STOP_SCL_HIGH <- STOP_SDA_LOW <- STOP_SCL_LOW
      ^                                     ^
      |                                     |
+-----+----- done -----+-----+

PREPARE_DATA_LOW -> SEND_BIT_HIGH
SEND_BIT_HIGH --más bits--> PREPARE_DATA_LOW
SEND_BIT_HIGH --último bit--> ACK_LOW -> ACK_HIGH

ACK_HIGH --ACK error--> STOP_SCL_LOW
ACK_HIGH --addr write y tx0--> PREPARE_DATA_LOW
ACK_HIGH --tx0 y tx_count=2--> PREPARE_DATA_LOW
ACK_HIGH --fin escritura--> STOP_SCL_LOW
ACK_HIGH --addr read--> READ_BIT_LOW -> READ_BIT_HIGH

READ_BIT_HIGH --más bits--> READ_BIT_LOW
READ_BIT_HIGH --último bit--> READ_NACK_LOW -> READ_NACK_HIGH -> STOP_SCL_LOW

```

Tabla de estados

Estado	Qué hace	Entrada	Salida	Señales activadas
IDLE	Bus libre. Espera orden.	Reset o fin anterior.	start='1'.	SCL='1', SDA liberada, busy='0'.
START_SDA_LOW	Inicia START bajando SDA con SCL alto.	IDLE.	Tick siguiente.	sda_drive_low='1'.
START_SCL_LOW	Baja SCL.	START_SDA_LOW.	Tick siguiente.	SCL='0'.
PREPARE_DATA_LOW	Coloca el bit a transmitir mientras SCL=0.	Inicio de byte o bit siguiente.	Tick siguiente.	SDA baja si bit es 0; liberada si bit es 1.
SEND_BIT_HIGH	Sube SCL para que el esclavo muestree.	PREPARE_DATA_LOW.	A ACK_LOW si bit 0; si no, vuelve a preparar.	SCL='1'.
ACK_LOW	Libera SDA antes del ACK.	Fin de byte.	Tick siguiente.	SCL='0', SDA liberada.
ACK_HIGH	Lee ACK. Decide próximo byte, lectura o STOP.	ACK_LOW.	Según ACK, fase y rw.	SCL='1', posible ack_error.
READ_BIT_LOW	Prepara lectura con SCL=0.	Lectura activa.	Tick siguiente.	SDA liberada.
READ_BIT_HIGH	Lee un bit desde SDA.	READ_BIT_LOW.	Más bits o NACK.	SCL='1', escribe rx_reg(bit_index).

Estado	Qué hace	Entrada	Salida	Señales activadas
READ_NACK_LOW	Prepara NACK final.	Último bit leído.	Tick siguiente.	SCL='0', SDA liberada.
READ_NACK_HIGH	Envía NACK manteniendo SDA liberada.	READ_NACK_LOW.	STOP.	SCL='1'.
STOP_SCL_LOW	Inicio de STOP.	Fin normal o error.	Tick siguiente.	SCL='0'.
STOP_SDA_LOW	Fuerza SDA baja.	STOP_SCL_LOW.	Tick siguiente.	SDA baja.
STOP_SCL_HIGH	Sube SCL con SDA baja.	STOP_SDA_LOW.	Tick siguiente.	SCL='1', SDA baja.
STOP_RELEASE	Libera SDA, genera done.	STOP_SCL_HIGH.	WAIT_START_RELEASE.	done='1', busy='0'.
WAIT_START_RELEASE	Evita retrigger mientras start siga alto.	STOP_RELEASE.	start='0'.	Bus libre.

Señales internas importantes

Señal	Importancia
state	FSM principal.
phase	Distingue dirección, tx_byte0, tx_byte1 y lectura.
tick_counter	Divide clk para temporizar el bus I2C.
sda_drive_low	Implementa open-drain: solo conduce 0 o libera la línea.
shift_reg	Byte actualmente transmitido.
bit_index	Bit actual, de 7 a 0.
ack_error_reg	Latch de error de ACK.

Riesgos o detalles delicados

- El módulo solo soporta escritura de 1 o 2 bytes y lectura de 1 byte. No implementa ráfagas generales ni repeated-start.
- SDA necesita pull-up externo o constraint/pull-up adecuado.
- El cálculo de TICK_DIVIDER usa división entera.
- WAIT_START_RELEASE es importante: si el controlador mantiene start alto, evita disparos repetidos.

Señales para mirar en waveform

state, phase, start, busy, done, ack_error, SCL, SDA, sda_drive_low, shift_reg, bit_index, tx_byte0, tx_byte1, tx_count.

3. Módulo uart_tx

Archivo: Protocolo de comunicacion/rtl/uart_tx.vhd

Función general

Transmisor UART 8N1: un bit de start, 8 bits de datos LSB-first y un bit de stop. No recibe datos; solo transmite.

Lugar dentro del sistema

UART. Recibe bytes desde frame_uart_dump y los saca por uart_tx.

Entradas principales

Señal	Descripción
clk	Reloj del sistema.
rst	Reset síncrono.
tx_start	Pulso para iniciar envío de tx_data.
tx_data	Byte a transmitir.

Salidas principales

Señal	Descripción
tx_line	Línea UART. Reposo en 1.
busy	Transmisión activa.
done	Pulso de un ciclo al terminar el stop bit.

Generics importantes

Generic	Valor por defecto	Uso
CLK_FREQ_HZ	100000000	Frecuencia de clk.
BAUD_RATE	921600	Velocidad UART.

Dominio de reloj

Todo el módulo trabaja en clk.

Reset

Con rst='1': vuelve a IDLE, tx_line='1', busy='0', limpia contador, índice y registro de desplazamiento.

Funcionamiento paso a paso

1. En IDLE, mantiene tx_line='1'.
2. Si tx_start='1', copia tx_data a shift_reg, baja tx_line y entra al bit de start.
3. Espera BAUD_DIVIDER ciclos por cada bit.
4. Envía bits de datos desde shift_reg(0) hasta shift_reg(7).
5. Envía stop bit en 1.
6. Pulsa done y vuelve a IDLE.

Diagrama de estados

```

IDLE --tx_start--> START_BIT -> DATA_BITS -> STOP_BIT -> IDLE
                ^           |
                |           +--- bit_index 0..7
                +--- espera BAUD_MAX en cada bit

```

Tabla de estados

Estado	Qué hace	Entrada	Salida	Señales activadas
IDLE	Línea en reposo. Espera byte.	Reset o fin.	tx_start='1'.	tx_line='1', busy='0'.
START_BIT	Mantiene bit de start.	Inicio de transmisión.	baud_counter=BAUD_MAX.	tx_line='0', busy='1'.
DATA_BITS	Envía 8 bits LSB-first.	Fin de start.	Tras bit 7.	tx_line=shift_reg(bit_index).

Estado	Qué hace	Entrada	Salida	Señales activadas
STOP_BIT	Envía stop bit.	Fin de datos.	baud_counter=BAUD_MAX.	tx_line='1', done='1' al final.

Señales internas importantes

state, baud_counter, bit_index, shift_reg, tx_reg, busy_reg, done_reg.

Riesgos o detalles delicados

- BAUD_DIVIDER es división entera; si la relación reloj/ baud no es exacta aparece pequeño error de baud.
- tx_start se atiende solo en IDLE. Si llega mientras busy='1', se ignora.
- El protocolo es solo TX, sin control de flujo.

Señales para mirar en waveform

tx_start, tx_data, tx_line, busy, done, state, baud_counter, bit_index.

4. Módulo ov7670_init_config

Archivo: Configuracion_camara/rtl/ov7670_init_config.vhd

Función general

Secuenciador de configuración inicial de la OV7670. Recorre una ROM de pares (registro, dato) y pide al i2c_master que escriba cada par. Después del primer registro de reset espera RESET_DELAY_MS.

Lugar dentro del sistema

Configuración cámara. Genera transacciones SCCB/I2C hacia i2c_master.

Entradas principales

Señal	Descripción
clk	Reloj de sistema.
rst	Reset síncrono.
start	Orden de iniciar la configuración.
i2c_done	El maestro terminó una escritura.
i2c_ack_error	El maestro detectó error de ACK.

Salidas principales

Señal	Descripción
i2c_start	Pulso para disparar el i2c_master.
i2c_rw	Siempre 0 para escritura.
i2c_tx_byte0	Dirección de registro OV7670.
i2c_tx_byte1	Valor a escribir.
i2c_tx_count	Siempre 2 en esta configuración.
busy	Secuencia en curso.
done	Configuración terminada.
error	Error de ACK.
current_step	Índice actual de la ROM.

Generics importantes

Generic	Valor por defecto	Uso
CLK_FREQ_HZ	1000000000	Para calcular espera tras reset.
RESET_DELAY_MS	1	Espera después de (0x12, 0x80).

Dominio de reloj

Todo trabaja en clk.

Reset

Vuelve a IDLE, pone config_index=0, delay_count=0, limpia salidas I2C, done y error.

Funcionamiento paso a paso

1. En IDLE, espera start='1'.
2. Carga el par actual de CONFIG_ROM.
3. Pulsa i2c_start.
4. Espera i2c_done.
5. Si hay i2c_ack_error, pasa a error.
6. Si el índice es 0, espera RESET_DELAY_MS porque el primer comando reinicia la cámara.
7. Avanza al siguiente par.
8. Al terminar la ROM, levanta done y espera que start baje.

La ROM tiene 21 pares. Incluye reset, control básico, formato YUV, escalado QQVGA, orden de bytes y ventana activa.

Diagrama de estados

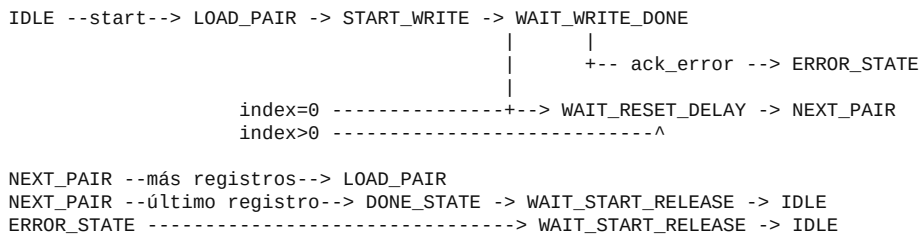


Tabla de estados

Estado	Qué hace	Entrada	Salida	Señales activadas
IDLE	Espera inicio.	Reset o fin.	start='1'.	done='0', error='0'.
LOAD_PAIR	Carga registro y valor desde ROM.	Inicio o siguiente par.	Siguiente ciclo.	i2c_rw='0', tx_count=2.
START_WRITE	Dispara el maestro I2C.	Par cargado.	Siguiente ciclo.	i2c_start='1'.
WAIT_WRITE_DONE	Espera fin de escritura.	START_WRITE.	i2c_done='1'.	Mantiene datos I2C.
WAIT_RESET_DELAY	Espera tras reset de cámara.	Después del primer par.	delay_count=RESET_DELAY_CYCLES.	Incrementa delay_count.

Estado	Qué hace	Entrada	Salida	Señales activadas
NEXT_PAIR	Decide avanzar o terminar.	Escritura válida.	Más pares o fin.	Incrementa config_index.
DONE_STATE	Marca configuración completa.	Último par escrito.	WAIT_START_RELEASE.	done= '1'.
ERROR_STATE	Marca error.	ACK fallido.	WAIT_START_RELEASE.	error= '1'.
WAIT_START_RELEASE	Espera que start baje.	Done o error.	start= '0'.	Mantiene done/error según origen.

Señales internas importantes

CONFIG_ROM, CONFIG_LEN, RESET_DELAY_CYCLES, config_index, delay_count, i2c_start_reg, done_reg, error_reg.

Riesgos o detalles delicados

- La configuración asume la dirección OV7670 usada en el top y que el maestro escribe dos bytes.
- Si start queda alto, el módulo queda en WAIT_START_RELEASE después de terminar.
- done queda alto en DONE_STATE; el top lo latched.
- La secuencia configura YUV/QQVGA, pero la validación final depende también de las señales reales de cámara.

Señales para mirar en waveform

state, config_index, current_step, i2c_start, i2c_tx_byte0, i2c_tx_byte1, i2c_done, i2c_ack_error, done, error.

5. Módulo ov7670_xclk_gen

Archivo: Configuración_camara/rtl/ov7670_xclk_gen.vhd

Función general

Genera xclk para la cámara usando una primitiva Xilinx MMCME2_BASE. Parte de clk_100mhz y produce aproximadamente 24 MHz.

Lugar dentro del sistema

Configuración / reloj de cámara. Entrega cam_xclk hacia la OV7670.

Entradas principales

Señal	Descripción
clk_100mhz	Reloj base de FPGA.
rst	Reset de la MMCM.

Salidas principales

Señal	Descripción
xclk	Reloj externo para OV7670.
locked	Indica que la MMCM está bloqueada.

Generics/parámetros importantes

No tiene generics de entidad. Los parámetros importantes están dentro del `generic map` de la MMCM:

Parámetro MMCM	Valor	Efecto
CLKIN1_PERIOD	10.0	Entrada de 100 MHz.
CLKFBOUT_MULT_F	12.0	Multiplicación interna.
CLKOUT0_DIVIDE_F	50.0	División de salida.
CLKOUT0_DUTY_CYCLE	0.5	50% duty cycle.

Con 100 MHz, la salida es $100 * 12 / 50 = 24$ MHz.

Dominio de reloj

No usa una FSM. La primitiva genera un dominio `xc1k`. En el top, `cam_xc1k` se usa como salida hacia la cámara, no como reloj interno de captura.

Reset

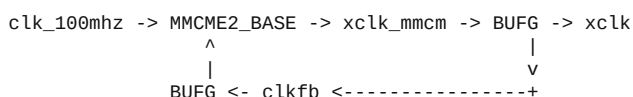
rst entra a RST de la MMCM. Al resetear, la MMCM pierde lock y vuelve a estabilizarse.

Funcionamiento paso a paso

1. `clk_100mhz` entra a `MMCME2_BASE`.
2. La MMCM genera `clkfb` para realimentación.
3. `clkfb` pasa por `BUFG` y vuelve a `CLKFBIN`.
4. `CLKOUT0` genera `xcclk_mmc`.
5. `xcclk_mmc` pasa por `BUFG`.
6. La salida final es `xcclk`.
7. `locked` sube cuando la MMCM está estable.

Diagrama

No tiene FSM explícita. Diagrama conceptual:



locked <= LOCKED de la MMCM

Tabla conceptual

Etapas	Qué hace	Entrada	Salida	Señales activadas
Entrada MMCM	Recibe 100 MHz.	clk_100mhz.	Reloj interno.	CLKIN1.
Realimentación	Estabiliza la MMCM.	clkfb.	clkfb_buf.	BUFG feedback.
Salida	Divide a 24 MHz.	MMCM interna.	clk_mmcm.	CLKOUT0.
Buffer salida	Lleva reloj a red global.	clk_mmcm.	clk.	BUFG.
Lock	Indica estabilidad.	MMCM.	locked.	LOCKED.

Señales internas importantes

clkfb, clkfb_buf, xclk_mmcm, locked.

Riesgos o detalles delicados

- Usa primitivas Xilinx; no es VHDL portable fuera de Vivado/Xilinx.
- locked debe considerarse antes de iniciar configuración en hardware.
- cam_xclk no debe confundirse con cam_pclk: xclk se entrega a la cámara; pclk vuelve desde la cámara con los datos.

Señales para mirar en waveform

clk_100mhz, rst, xclk, locked.

6. Módulo ov7670_capture_y_stream

Archivo: Configuracion_camara/rtl/ov7670_capture_y_stream.vhd

Función general

Captura el bus paralelo de la OV7670 en flanco ascendente de pclk. Interpreta bytes en orden Y U Y V y emite solo los bytes Y con coordenadas x/y.

Lugar dentro del sistema

Captura. Es el primer bloque que procesa video real de la cámara.

Entradas principales

Señal	Descripción
pclk	Reloj de pixel entregado por la cámara.
rst	Reset del capturador.
enable	Habilita captura.
vsync	Sincronismo vertical de cámara.
href	Indica datos válidos de línea.
data	Byte paralelo de cámara.

Salidas principales

Señal	Descripción
pixel_y	Byte de luminancia capturado.
pixel_valid	Pulso cuando pixel_y es válido.
pixel_x	Columna del pixel Y.
pixel_y_pos	Fila del pixel Y.
frame_active	Indica frame en curso.
frame_done	Pulso cuando termina un frame activo por vsync.
overflow	Error por exceder tamaño configurado.

Generics importantes

Generic	Valor por defecto	Uso
FRAME_WIDTH	160	Columnas esperadas.
FRAME_HEIGHT	120	Filas esperadas.
VSYNC_ACTIVE_LEVEL	'1'	Nivel activo de VSYNC.
HREF_ACTIVE_LEVEL	'1'	Nivel activo de HREF.

Dominio de reloj

Todo trabaja en cam_pclk mediante el puerto pclk.

Reset

Limpia contadores x_count, y_count, byte_phase, href_active_prev, frame_active, registros de pixel y overflow.

Funcionamiento paso a paso

1. Cada flanco de pclk baja por defecto pixel_valid y frame_done.
2. Si enable='0', reinicia contadores y no captura.
3. Si vsync está activo, considera que no hay frame útil. Si venía de un frame activo, emite frame_done.
4. Si vsync no está activo, marca frame_active='1'.
5. Si href está inactivo, reinicia x_count y byte_phase. Si acaba de terminar una línea (href pasó de 1 a 0), incrementa y_count.
6. Si href está activo, consume bytes.
7. byte_phase=0 y byte_phase=2 son bytes Y; se emiten como pixel_y.
8. byte_phase=1 y byte_phase=3 son U/V; se ignoran.
9. byte_phase rota 0,1,2,3,0...
10. Si se exceden FRAME_WIDTH o FRAME_HEIGHT, activa overflow.

Diagrama

No tiene FSM explícita. Diagrama conceptual:

```
enable=0
|
v
reset contadores internos

enable=1
|
+-- vsync activo --> fin/no frame; si frame_active previo: frame_done=1
|
+-- vsync inactivo --> frame_active=1
|
|   +-- href=0 --> x_count=0, byte_phase=0,
|   |               si flanco de bajada HREF: y_count++
|   |
|   |   +-- href=1 --> leer data según byte_phase
|   |   |               0:Y emitir pixel
|   |   |               1:U ignorar
|   |   |               2:Y emitir pixel
|   |   |               3:V ignorar
```

Tabla conceptual de fases

Fase	Qué hace	Condición de entrada	Condición de salida	Señales activadas
Deshabilitado	Limpia captura.	enable='0'.	enable='1'.	frame_active='0'.
VSynC activo	Termina o espera frame.	vsync_active='1'.	vsync_active='0'.	frame_done='1' si venía activo.
Fuera de línea	Espera HREF.	href_active='0'.	href_active='1'.	Reinicia x_count, byte_phase.
Byte Y0	Captura luminancia.	href_active='1', byte_phase=0.	Siguiente byte.	pixel_valid='1', x_count++.
Byte U	Ignora crominancia.	byte_phase=1.	Siguiente byte.	Ninguna salida de pixel.
Byte Y1	Captura luminancia.	byte_phase=2.	Siguiente byte.	pixel_valid='1', x_count++.

Fase	Qué hace	Condición de entrada	Condición de salida	Señales activadas
Byte V	Ignora crominancia.	byte_phase=3.	Vuelve a fase 0.	Ninguna salida de pixel.

Señales internas importantes

x_count, y_count, byte_phase, href_active, vsync_active, href_active_prev, frame_active_reg, pixel_valid_reg, frame_done_reg, overflow_reg.

Riesgos o detalles delicados

- La lógica asume orden Y U Y V.
- frame_done depende de que vsync vuelva al nivel activo después de haber estado en frame.
- Si vsync aparece temprano, el módulo puede marcar fin de frame aunque no hayan llegado todos los píxeles; el control superior debe validar el flujo real.
- pixel_valid es un pulso de un ciclo de pclk.
- El incremento de fila ocurre al detectar fin de href.

Señales para mirar en waveform

pclk, enable, vsync, href, data, byte_phase, x_count, y_count, pixel_y, pixel_valid, pixel_x, pixel_y_pos, frame_done, overflow.

7. Módulo frame_capture_store

Archivo: Configuración_camara/rtl/frame_capture_store.vhd

Función general

Recibe píxeles Y válidos con coordenadas, calcula la dirección lineal en memoria y genera las escrituras hacia la BRAM. Congela la captura cuando termina el frame y espera a que el bloque UART indique que ya terminó de leer/dumpear.

Lugar dentro del sistema

RAM / almacenamiento de captura.

Entradas principales

Señal	Descripción
pclk	Reloj de captura.
rst	Reset.
arm_capture	Pulso para iniciar captura.
dump_done_toggle	Toggle que indica que UART terminó de enviar el frame anterior.
pixel_y	Byte de luminancia.
pixel_valid	Pulso de pixel válido.
pixel_x, pixel_y_pos	Coordenadas del pixel.
frame_done	Fin de frame desde el capturador.

Salidas principales

Señal	Descripción
wr_en	Pulso de escritura a BRAM.
wr_addr	Dirección lineal.

Señal	Descripción
wr_data	Dato Y a escribir.
capture_busy	Captura activa.
frame_ready	Frame completo listo para lectura.
frame_ready_toggle	Toggle para avisar al dominio clk.
overflow	Coordenada o dirección fuera de rango.

Generics importantes

Generic	Valor por defecto	Uso
FRAME_WIDTH	160	Columnas.
FRAME_HEIGHT	120	Filas.
ADDR_WIDTH	15	Bits de dirección para 19200 bytes.

Dominio de reloj

Todo trabaja en cam_pclk mediante pclk.

Reset

Vuelve a IDLE, limpia registros de escritura, frame_ready, toggle, overflow y captura el valor actual de dump_done_toggle.

Funcionamiento paso a paso

1. En IDLE, espera arm_capture='1'.
2. En CAPTURING, por cada pixel_valid='1' calcula pixel_addr = y * FRAME_WIDTH + x.
3. Si coordenadas y dirección están dentro de rango, genera wr_en, wr_addr y wr_data.
4. Si algo está fuera de rango, activa overflow.
5. Cuando llega frame_done='1', activa frame_ready, cambia frame_ready_toggle y entra a READY.
6. En READY, no escribe más y espera dump_done_toggle distinto al anterior.
7. Cuando detecta el toggle de dump terminado, vuelve a IDLE y baja frame_ready.

Diagrama de estados

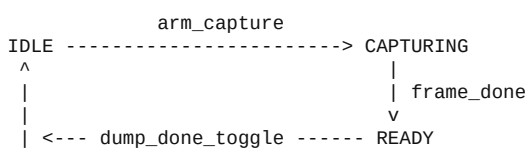


Tabla de estados

Estado	Qué hace	Entrada	Salida	Señales activadas
IDLE	Espera armado de captura.	Reset o dump terminado.	arm_capture='1'.	capture_busy='0', limpia overflow.
CAPTURING	Escribe píxeles Y en BRAM.	arm_capture.	frame_done='1'.	capture_busy='1', wr_en en pixel válido.
READY	Frame congelado/listo.	Fin de frame.	Cambio de dump_done_toggle.	frame_ready='1'.

Señales internas importantes

state, FRAME_PIXELS, wr_en_reg, wr_addr_reg, wr_data_reg, frame_ready_reg, frame_ready_toggle_reg, overflow_reg, dump_done_toggle_prev.

Riesgos o detalles delicados

- Usa multiplicación y $\ast$ FRAME_WIDTH para dirección; Vivado puede implementarla como lógica aritmética.
- No valida que se hayan recibido exactamente FRAME_PIXELS antes de frame_done; confía en el capturador y en el timing de cámara.
- Mientras está en READY, no rearma hasta recibir dump_done_toggle.
- dump_done_toggle viene de otro dominio, pero en el top se sincroniza antes de entrar aquí.

Señales para mirar en waveform

state, arm_capture, pixel_valid, pixel_x, pixel_y_pos, pixel_y, wr_en, wr_addr, wr_data, frame_done, frame_ready, frame_ready_toggle, dump_done_toggle, overflow.

8. Módulo framebuffer_y_bram

Archivo: Configuración camara/rtl/framebuffer_y_bram.vhd

Función general

Memoria de frame para luminancia Y. Tiene puerto de escritura en el reloj de cámara y puerto de lectura en el reloj del sistema.

Lugar dentro del sistema

RAM / framebuffer.

Entradas principales

Señal	Descripción
wr_clk	Reloj de escritura, normalmente cam_clk.
wr_en	Habilita escritura.
wr_addr	Dirección de escritura.
wr_data	Dato Y a guardar.
rd_clk	Reloj de lectura, normalmente clk.
rd_addr	Dirección de lectura.

Salidas principales

Señal	Descripción
rd_data	Dato leído. Tiene registro de salida.

Generics importantes

Generic	Valor por defecto	Uso
ADDR_WIDTH	15	Dirección suficiente para 19200 posiciones.
DATA_WIDTH	8	Un byte Y.
FRAME_PIXELS	19200	Profundidad real.

Dominio de reloj

Dual-clock:

- Escritura en `wr_clk` (`cam_pclk` en el top).
- Lectura en `rd_clk` (`clk` en el top).

Reset

No tiene reset explícito. La RAM se inicializa a cero en la declaración: `ram := (others => (others => '0'))`. En hardware, la inferencia exacta depende de Vivado y del tipo de BRAM inferida.

Funcionamiento paso a paso

1. En flanco de `wr_clk`, si `wr_en='1'` y `wr_addr < FRAME_PIXELS`, escribe `wr_data`.
2. En flanco de `rd_clk`, si `rd_addr < FRAME_PIXELS`, copia `ram(rd_addr)` a `rd_data_reg`.
3. Si la dirección de lectura está fuera de rango, entrega cero.
4. `rd_data` sale desde el registro `rd_data_reg`, por lo que hay latencia de lectura síncrona.

Diagrama

No tiene FSM explícita. Diagrama conceptual:

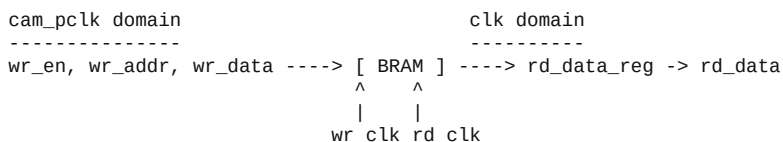


Tabla conceptual

Etapas	Qué hace	Condición de entrada	Condición de salida	Señales activadas
Escritura	Guarda pixel Y.	<code>rising_edge(wr_clk)</code> y <code>wr_en='1'</code> .	Siguiente flanco.	Actualiza <code>ram(wr_addr)</code> .
Lectura válida	Lee una dirección.	<code>rising_edge(rd_clk)</code> y dirección válida.	Siguiente ciclo visible.	Actualiza <code>rd_data_reg</code> .
Lectura fuera de rango	Protege salida.	<code>rd_addr >= FRAME_PIXELS</code> .	Siguiente ciclo.	<code>rd_data_reg=0</code> .

Señales internas importantes

`ram`, `rd_data_reg`.

Riesgos o detalles delicados

- Es una RAM dual-clock inferida; conviene revisar inferencia de Vivado si se cambia el estilo.
- La lectura es síncrona: el consumidor debe esperar un ciclo luego de poner `rd_addr`. `frame_uart_dump` incluye `WAIT_PAYLOAD_DATA`.
- No hay doble buffer; durante el dump no se debe sobrescribir la BRAM. Eso lo controla `frame_capture_store`.

Señales para mirar en waveform

`wr_clk`, `wr_en`, `wr_addr`, `wr_data`, `rd_clk`, `rd_addr`, `rd_data`.

9. Módulo `frame_uart_dump`

Archivo: Configuración `camara/rtl/frame_uart_dump.vhd`

Función general

Cuando detecta un nuevo frame listo, lee la BRAM secuencialmente y transmite por UART un paquete binario: encabezado, payload de luminancia y checksum.

Lugar dentro del sistema

UART / lectura de framebuffer.

Entradas principales

Señal	Descripción
clk	Reloj del sistema.
rst	Reset.
frame_ready_toggle	Toggle sincronizado que indica frame listo.
rd_data	Dato leído desde BRAM.
uart_busy	UART ocupado.
uart_done	UART terminó byte actual.

Salidas principales

Señal	Descripción
rd_addr	Dirección de BRAM a leer.
uart_start	Pulso para iniciar transmisión de un byte.
uart_data	Byte entregado a UART.
dump_busy	Dump en curso.
dump_done	Pulso de fin de dump.
dump_done_toggle	Toggle para avisar al dominio cam_clk.

Generics importantes

Generic	Valor por defecto	Uso
FRAME_WIDTH	160	Se codifica en header.
FRAME_HEIGHT	120	Se codifica en header.
ADDR_WIDTH	15	Ancho de dirección BRAM.

$\text{FRAME_PIXELS} = \text{FRAME_WIDTH} * \text{FRAME_HEIGHT}.$

Dominio de reloj

Todo trabaja en clk.

Reset

Vuelve a IDLE, captura el toggle actual como previo, limpia índices, dirección, dato UART, checksum, busy y toggle de done.

Funcionamiento paso a paso

1. En IDLE, espera que frame_ready_toggle cambie.
2. Al detectar un nuevo frame, limpia header_index, payload_index, checksum y entra a enviar header.
3. Envía 12 bytes de header:
 - 0x53 0x54 0x59 0x31 = STY1
 - width little-endian
 - height little-endian
 - payload length little-endian de 32 bits

- 4. Para payload, pone rd_addr = payload_index.
- 5. Espera un ciclo para que rd_data sea válido.
- 6. Envía rd_data por UART y acumula checksum módulo 256.
- 7. Repite hasta FRAME_PIXELS bytes.
- 8. Envía checksum final.
- 9. Pulsa dump_done, cambia dump_done_toggle y vuelve a IDLE.

Diagrama de estados

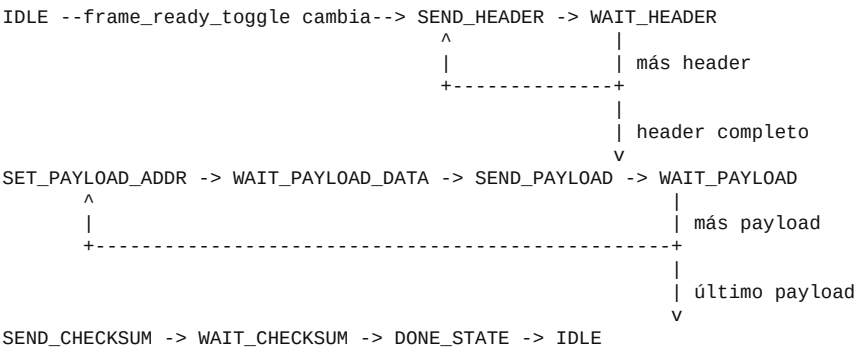


Tabla de estados

Estado	Qué hace	Entrada	Salida	Señales activadas
IDLE	Espera frame listo.	Reset o dump terminado.	Cambio de frame_ready_toggle.	dump_busy='0'.
SEND_HEADER	Presenta byte de header a UART.	Nuevo frame o header siguiente.	uart_busy='0'.	uart_data=HEADER(i), uart_start='1'.
WAIT_HEADER	Espera fin de byte header.	SEND_HEADER.	uart_done='1'.	dump_busy='1'.
SET_PAYLOAD_ADDR	Coloca dirección de BRAM.	Header completo o byte siguiente.	Siguiente ciclo.	rd_addr=payload_index.
WAIT_PAYLOAD_DATA	Espera latencia de BRAM.	Dirección colocada.	Siguiente ciclo.	dump_busy='1'.
SEND_PAYLOAD	Envía rd_data.	Dato listo.	uart_busy='0'.	uart_data=rd_data, uart_start='1', checksum suma.
WAIT_PAYLOAD	Espera fin de byte payload.	SEND_PAYLOAD.	uart_done='1'.	Incrementa payload_index o va a checksum.
SEND_CHECKSUM	Envía checksum final.	Payload completo.	uart_busy='0'.	uart_data=checksum, uart_start='1'.
WAIT_CHECKSUM	Espera fin del checksum.	SEND_CHECKSUM.	uart_done='1'.	dump_busy='1'.
DONE_STATE	Avisa fin de dump.	Checksum enviado.	Siguiente ciclo.	dump_done='1', toggle de done.

Señales internas importantes

state, HEADER, ready_toggle_prev, header_index, payload_index, rd_addr_reg, uart_start_reg, uart_data_reg, checksum_reg, dump_done_toggle_reg.

Riesgos o detalles delicados

- Depende de la latencia de lectura de BRAM; por eso existe WAIT_PAYLOAD_DATA.

- `frame_ready_toggle` debe venir sincronizado al dominio `clk`.
- No tiene control de pausa externo: al detectar frame listo, transmite todo.
- El checksum suma solo payload, no header.

Señales para mirar en waveform

`state`, `frame_ready_toggle`, `rd_addr`, `rd_data`, `payload_index`, `checksum_reg`, `uart_start`, `uart_data`, `uart_busy`, `uart_done`, `dump_busy`, `dump_done`, `dump_done_toggle`.

10. Módulo `star_tracker_top`

Archivo: `Top/rtl/star_tracker_top.vhd`

Función general

Integra todos los bloques: genera XCLK, configura la cámara por I2C, arma captura en `cam_pclk`, guarda un frame en BRAM, sincroniza el aviso al dominio `clk`, lee la BRAM y transmite el frame por UART.

Lugar dentro del sistema

Top.

Entradas principales

Señal	Descripción
<code>clk</code>	Reloj principal 100 MHz.
<code>rst</code>	Reset global.
<code>start_btn</code>	Botón/orden de inicio.
<code>SDA</code>	Línea I2C bidireccional.
<code>cam_pclk</code>	Pixel clock que viene de cámara.
<code>cam_vsync</code> , <code>cam_href</code>	Sincronismos de video.
<code>cam_data</code>	Bus paralelo de cámara.

Salidas principales

Señal	Descripción
<code>SCL</code>	Reloj I2C.
<code>cam_xclk</code>	Reloj 24 MHz hacia cámara.
<code>cam_pwn</code>	Power down de cámara, fijo en 0.
<code>cam_reset</code>	Reset de cámara, not <code>rst</code> .
<code>uart_tx</code>	Línea UART TX.
<code>pixel_valid</code> , <code>frame_done</code>	Señales de debug desde captura.
<code>led_read_data</code>	Último pixel Y capturado.
<code>busy</code> , <code>ok</code> , <code>fail</code>	Estado general/debug.

Generics importantes

Generic	Valor por defecto	Uso
<code>FRAME_WIDTH</code>	160	Ancho del frame.
<code>FRAME_HEIGHT</code>	120	Alto del frame.
<code>ADDR_WIDTH</code>	15	Dirección BRAM.

Generic	Valor por defecto	Uso
UART_BAUD_RATE	921600	Baud UART.
SIM_BYPASS_XCLK_LOCK	false	Solo para simulación: permite iniciar sin esperar lock real.

Dominio de reloj

Tiene tres dominios relevantes:

Dominio	Bloques/señales
clk	ov7670_init_config, i2c_master, latches de init/error/dump, sincronización de frame_ready_toggle, frame_uart_dump, uart_tx, lectura BRAM.
cam_pclk	Sincronización de init/start/dump hacia captura, ov7670_capture_y_stream, frame_capture_store, escritura BRAM.
cam_xclk	Generado por ov7670_xclk_gen y enviado a cámara. No clockea lógica interna del top.

Reset

rst entra a todos los bloques principales. Además:

- cam_reset <= not rst.
- cam_pwdn <= '0'.
- Latches init_done_latched, init_error_latched, dump_done_latched se limpian en clk.
- Sincronizadores de cam_pclk se limpian en cam_pclk.

Funcionamiento paso a paso

1. ov7670_xclk_gen genera cam_xclk y xclk_locked.
2. xclk_ready vale xclk_locked, salvo si SIM_BYPASS_XCLK_LOCK=true.
3. init_start = start_btn and xclk_ready and not init_done_latched.
4. ov7670_init_config recorre la ROM y pide escrituras al i2c_master.
5. Cuando init_done sube, el top lo guarda en init_done_latched.
6. init_done_latched cruza al dominio cam_pclk con doble FF.
7. Al detectar init lista, o un nuevo flanco de start_btn sincronizado a cam_pclk, genera capture_arm_pulse.
8. frame_capture_store entra a CAPTURING; entonces store_capture_busy='1'.
9. capture_rst = rst or not init_done_pclk or not store_capture_busy, por lo que el capturador solo corre durante captura armada.
10. ov7670_capture_y_stream extrae bytes Y.
11. frame_capture_store escribe esos Y en BRAM.
12. Cuando llega frame_done, frame_capture_store activa store_frame_ready y cambia store_frame_ready_toggle.
13. store_frame_ready_toggle cruza a clk con doble FF.
14. frame_uart_dump detecta el toggle, lee BRAM y manda bytes a uart_tx.
15. Al terminar, frame_uart_dump cambia dump_done_toggle.
16. dump_done_toggle cruza a cam_pclk y libera frame_capture_store para una futura captura.
17. ok = init_done_latched and dump_done_latched.

18. fail = init_error_latched or capture_overflow or store_overflow.

Diagrama de integración de módulos



Diagrama general del flujo completo

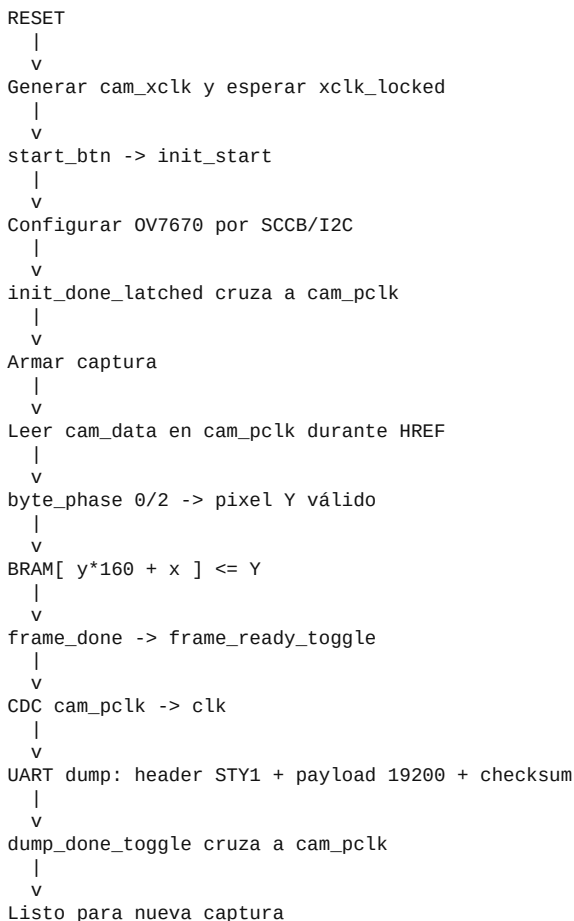
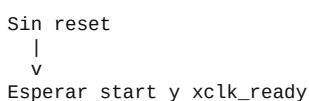


Diagrama de estados

star_tracker_top no tiene una FSM explícita propia. Su comportamiento sale de FSMs internas y señales de armado. Diagrama conceptual:



```

|
v
Init I2C activa
|
v
Init latcheada
|
v
Captura activa en cam_pclk
|
v
Frame ready
|
v
Dump UART activo
|
v
Dump done / ok

```

Tabla conceptual del top

Etapas	Qué hace	Condición de entrada	Condición de salida	Señales activadas
Espera inicial	Mantiene sistema sin init.	Reset liberado.	start_btn and xclk_ready.	busy='0' salvo clocks.
Init cámara	Escribe ROM por I2C.	init_start='1'.	init_done o init_error.	init_busy, i2c_busy.
Armado captura	Cruza init a cam_pclk.	init_done_latched.	capture_arm_pulse.	Sincronizadores pclk.
Captura	Extrae Y y escribe BRAM.	store_capture_busy='1'.	capture_frame_done.	pixel_valid, fb_wr_en.
Frame listo	Congela BRAM.	frame_done.	frame_ready_toggle cruza a clk.	store_frame_ready.
Dump UART	Lee BRAM y transmite.	Toggle detectado en clk.	dump_done.	dump_busy, uart_start.
Listo/rearme	Permite nueva captura.	dump_done_toggle cruza a cam_pclk.	Nuevo start_btn.	ok='1'.

Tabla de cruces CDC

Señal	Origen	Destino	Mecanismo	Comentario
init_done_latched	clk	cam_pclk	Doble FF: init_done_pclk_meta, init_done_pclk.	Se usa para armar captura.
start_btn hacia captura	externo/top	cam_pclk	Doble FF: start_btn_pclk_meta, start_btn_pclk.	Detecta flanco para recaptura.
store_frame_ready_toggle	cam_pclk	clk	Doble FF: frame_ready_toggle_meta, frame_ready_toggle_sys.	Toggle seguro para evento de frame listo.
dump_done_toggle	clk	cam_pclk	Doble FF: dump_done_toggle_meta, dump_done_toggle_pclk.	Libera frame_capture_store.
BRAM write/read	cam_pclk	clk	RAM dual-clock.	Escritura por captura, lectura por UART.
capture_frame_done	cam_pclk	salida top	Sin sincronizar.	Es debug/salida, no controla UART en clk.
capture_pixel_valid	cam_pclk	salida top	Sin sincronizar.	Debug/salida.

Señal	Origen	Destino	Mecanismo	Comentario
capture_overflow, store_overflow	cam_pclk	fail combi nacional	Sin sincronizar.	Adecuado para LED/debug, delicado si alimenta lógica síncrona.
start_btn para init_start	externo	clk	Uso directo combinacional.	Riesgo: debería sincronizarse/debouncearse para hardware robusto.
xclk_locked para init_start	MMCM/clk-related	clk	Uso directo.	Normalmente LOCKED es estable, pero puede sincronizarse si se quiere máxima robustez.

Señales internas importantes

xclk_locked, xclk_ready, init_start, init_done_latched, init_error_latched, init_done_pclk, capture_arm_pulse, capture_rst, fb_wr_en, fb_wr_addr, fb_wr_data, store_frame_ready_toggle, frame_ready_toggle_sys, dump_done_toggle, dump_done_latched, uart_start, uart_data, uart_busy, uart_done.

Riesgos o detalles delicados

- start_btn se usa directo para generar init_start en clk; para hardware real conviene sincronizador/debounce en clk.
- busy y fail mezclan señales de dominios distintos. Como salidas de estado/debug está bien, pero no deberían alimentar lógica crítica sin sincronización.
- capture_rst depende de señales en cam_pclk; su intención es mantener el capturador limpio hasta que frame_capture_store esté capturando.
- SIM_BYPASS_XCLK_LOCK debe mantenerse false para hardware normal.
- ok solo sube después de init y dump UART completo, no justo después de capturar.

Señales para mirar en waveform

Sistema: start_btn, xclk_locked, xclk_ready, init_start, init_done_latched, busy, ok, fail.

Captura: cam_pclk, cam_vsync, cam_href, cam_data, capture_arm_pulse, store_capture_busy, capture_pixel_valid, capture_pixel_y, capture_pixel_x, capture_pixel_y_pos, capture_frame_done.

BRAM/UART: fb_wr_en, fb_wr_addr, fb_wr_data, store_frame_ready_toggle, frame_ready_toggle_sys, fb_rd_addr, fb_rd_data, dump_busy, uart_start, uart_data, uart_busy, uart_done, dump_done_toggle.

11. Conclusión: recorrido completo de una imagen

El sistema arranca generando cam_xclk para la OV7670. Cuando el usuario presiona start_btn y la MMCM está lista, ov7670_init_config escribe por SCCB/I2C la ROM de configuración. Esa ROM pone la cámara en una configuración pensada para QQVGA 160x120, formato YUV 4:2:2 y orden Y U Y V.

Después de la configuración, el evento de init cruza al dominio cam_pclk. Allí se arma frame_capture_store, lo que habilita ov7670_capture_y_stream. El capturador mira VSYNC, HREF y cam_data en flancos de cam_pclk. Cada cuatro bytes del flujo Y U Y V, solo emite los dos bytes Y. Cada byte Y sale con pixel_valid, pixel_x y pixel_y_pos.

frame_capture_store convierte coordenadas a dirección lineal:

$addr = y * 160 + x$

y escribe ese byte en framebuffer_y_bram. Al terminar el frame, cambia frame_ready_toggle. Ese toggle cruza de cam_pclk a clk, donde frame_uart_dump empieza a leer la BRAM. El dump manda por UART:

STY1 + width + height + payload_length + 19200 bytes Y + checksum

Cuando UART termina, dump_done_toggle cruza de regreso a cam_pclk y libera el almacenador para una nueva captura.

La idea clave para entender el sistema es esta:

clk controla configuración y transmisión.

cam_pclk controla captura y escritura.

La BRAM une ambos mundos.

Los toggles sincronizados avisan eventos entre dominios.